UC | Grupo de Arquitectura y Tecnología de Computadores

# Práctica de Desarrollo de Aplicaciones Web

Aplicación web interactiva: diseño y desarrollo de un proyecto propio

G2105 – Desarrollo de Aplicaciones Web

15 de octubre de 2026

## Introducción

En esta práctica se desarrollará, por equipos, una aplicación web interactiva de temática libre. La idea es construir una herramienta que resulte útil o entretenida y que, al mismo tiempo, permita poner en práctica los conceptos fundamentales del desarrollo web: gestión de usuarios, operaciones CRUD, persistencia de datos, comunicación HTTP, validación y diseño de interfaces. Dada la temática libre, no se pretende que todas las aplicaciones tengan exáctamente las mismas funcionalidades. Sin embargo, sí deberán cumplir unos requisitos comunes que permitan evaluar de forma comparable los distintos proyectos. Estos requisitos vienen a responder a los conceptos fundamentales mencionados anteriormente.

La aplicación puede ser de carácter social, educativo, organizativo, creativo o lúdico. Por ejemplo, podrías ser una versión propia de Kahoot!, una plataforma para organizar viajes en grupo, una comunidad para compartir colecciones o un juego de navegador con progresión y competición entre usuarios. Al final de este documento se incluyen más ideas para inspirar la elección.

## Objetivos

Mediante esta práctica se pretende que el alumnado sea capaz de:

- Diseñar y desarrollar una aplicación web completa, desde el modelo de datos hasta la interfaz.
- Implementar operaciones de creación, consulta, modificación y eliminación (CRUD).
- Gestionar usuarios, autenticación, roles y permisos de acceso.
- Diseñar una interfaz accesible y adaptable a pantallas de escritorio y dispositivos móviles.
- Diseñar e implementar un contrato de comunicación entre cliente y servidor mediante HTTP.
- Validar los datos tanto en el cliente como en el servidor y tratar los errores de forma comprensible.
- Diseñar y automatizar pruebas que verifiquen el comportamiento de la aplicación.
- Justificar las decisiones de diseño e implementación y explicar individualmente el funcionamiento del sistema.

## Elección del proyecto

Cada equipo propondrá una aplicación y describirá su propósito, sus usuarios, las entidades principales y las operaciones que podrán realizar. Se valorará que la propuesta tenga una identidad clara y un flujo

de uso coherente. No es necesario desarrollar una idea completamente original: se puede partir de una aplicación conocida, siempre que se concrete qué problema resuelve, para quién y qué funcionalidades se implementarán.

Antes de comenzar el desarrollo, conviene definir:

- **Objetivo:** ¿para qué sirve la aplicación?
- **Usuarios y permisos:** ¿qué tipos de usuario existen y qué puede hacer cada uno?
- **Modelo de datos:** ¿cuáles son las entidades principales y cómo se relacionan?
- **Flujos principales:** ¿qué acciones debe poder completar un usuario de principio a fin?
- **Alcance:** ¿qué funcionalidades forman parte de la versión mínima y cuáles serían ampliaciones opcionales?

# Requisitos comunes

Los siguientes requisitos son aplicables a todos los proyectos, con independencia de su temática.

## Usuarios y control de acceso

- La aplicación permitirá identificar a sus usuarios mediante un mecanismo de autenticación.
- Los usuarios dispondrán de un perfil o espacio personal con información relevante para el dominio de la aplicación.
- Las acciones estarán sujetas a permisos y roles. Por ejemplo, un usuario cliente no podrá añadir productos al catálogo, y un usuario administrador sí podrá hacerlo.
- La autorización deberá comprobarse en el servidor; no basta con ocultar botones en la interfaz.

## Operaciones CRUD y modelo de datos

- La aplicación incluirá operaciones de creación, consulta, modificación y eliminación sobre sus entidades principales.
- El modelo de datos tendrá varias entidades relacionadas de forma significativa. No se considerará suficiente una aplicación que únicamente gestione una colección aislada de elementos sin relaciones ni reglas de negocio.
- Se utilizarán identificadores estables para los elementos y se mantendrá la integridad de las relaciones.
- Se implementarán validaciones para evitar datos incompletos, incoherentes o inválidos.
- Los datos relevantes persistirán entre sesiones y reinicios de la aplicación, salvo que la naturaleza de la propuesta justifique otra solución y se acuerde previamente.

## Comunicación cliente-servidor

- La comunicación entre el navegador y el servidor se realizará mediante el protocolo HTTP.
- No se permite utilizar WebSockets. La interacción en tiempo real, si la hubiera, deberá resolverse con mecanismos basados en HTTP, como peticiones periódicas (*polling*) o actualización manual.
- La aplicación deberá utilizar JavaScript en el navegador o en el entorno de ejecución que corresponda al diseño elegido. No se permite WebAssembly.

- Se podrá elegir libremente el lenguaje, las bibliotecas y los frameworks en el servidor, siempre que se respeten las restricciones anteriores y se puedan desplegar y ejecutar de forma reproducible.
- El servidor validará todas las operaciones recibidas, incluso si el cliente ya realiza validaciones.
- Los errores se comunicarán mediante respuestas HTTP adecuadas y mensajes comprensibles para el usuario. La interfaz no deberá asumir que todas las peticiones tienen éxito.

## Accesibilidad

La aplicación debe poder utilizarse por personas con distintas capacidades y formas de interacción. La accesibilidad debe considerarse desde el diseño inicial, no únicamente al final del desarrollo.

Como mínimo:

- Utilizar HTML semántico y controles apropiados para cada acción.
- Permitir la navegación y el uso de las funciones principales mediante teclado, con un orden de foco coherente y un indicador de foco visible.
- Asociar etiquetas a los campos de formulario y proporcionar instrucciones y mensajes de error claros.
- Mantener contraste suficiente entre el texto y el fondo; no utilizar únicamente el color para transmitir información.
- Proporcionar textos alternativos adecuados para las imágenes informativas y evitar alternativas redundantes en imágenes decorativas.
- Identificar correctamente el idioma de la página y la estructura de encabezados.
- Comunicar los cambios de estado relevantes (por ejemplo, confirmaciones, errores o resultados) de forma que puedan percibirse también con tecnologías de apoyo cuando corresponda.
- Evitar interacciones que dependan exclusivamente de arrastrar elementos, gestos complejos, sonido o animaciones.

Se recomienda utilizar como referencia las pautas WCAG 2.2 de nivel AA y comprobar la aplicación con navegación por teclado y herramientas de evaluación automática. Una puntuación automática favorable no sustituye a las comprobaciones manuales.

## Diseño adaptable: escritorio y móvil

La interfaz deberá ser usable tanto en pantallas de escritorio como en dispositivos móviles. No basta con que la página se reduzca para caber en una pantalla pequeña: el contenido y las interacciones deben seguir siendo comprensibles y manejables.

- El diseño se adaptará a distintos anchos de pantalla sin provocar desplazamiento horizontal innecesario.
- Los textos deberán seguir siendo legibles y los controles táctiles tendrán un tamaño y separación adecuados.
- Los formularios, tablas, menús, tarjetas y diálogos se adaptarán al espacio disponible.
- Las funciones principales estarán disponibles tanto con ratón como con teclado y pantalla táctil.
- Las imágenes y otros recursos visuales se escalarán correctamente y no deberán romper la distribución.
- Se comprobarán, como mínimo, una vista de escritorio y una vista móvil durante el desarrollo y antes de la entrega.

## Pruebas automatizadas

Las pruebas son parte del proyecto y se evalúan igual que el resto de requisitos. Su objetivo no es alcanzar un porcentaje de cobertura de 100 %, sino demostrar que las reglas más importantes de la aplicación se cumplen y que seguirán cumpliéndose cuando el código cambie.

Como mínimo, el proyecto incluirá **pruebas de API o de integración** que ejerciten la aplicación a través de HTTP, es decir, enviando peticiones al servidor y comprobando las respuestas y el estado resultante de los datos. Estas pruebas deberán cubrir:

- **Autenticación:** las operaciones que requieren identificación se rechazan cuando no hay una sesión válida.
- **Autorización:** un usuario no puede consultar, modificar ni eliminar contenido de otro usuario sin una regla explícita que lo permita. Debe comprobarse, al menos, un caso por cada tipo de permiso definido.
- **Operaciones CRUD:** creación, consulta, modificación y eliminación de las entidades principales, verificando que el cambio queda reflejado en los datos.
- **Validación:** el servidor rechaza datos incompletos, incoherentes o inválidos con un código de error HTTP adecuado, incluso si la petición no procede de la interfaz.
- **Reglas de negocio:** al menos las reglas más relevantes del dominio (por ejemplo, plazas disponibles, presupuesto máximo o estados permitidos), incluidos los casos límite.
- **Flujo principal:** el recorrido de principio a fin definido por el equipo.

Las pruebas deberán cumplir además estas condiciones:

- Se ejecutan con un único comando documentado en las instrucciones del proyecto.
- Son independientes entre sí y no dependen del orden de ejecución: cada prueba prepara los datos que necesita.
- Utilizan una base de datos o un almacenamiento de pruebas, nunca los datos reales ni de desarrollo.
- Comprueban resultados concretos (código de respuesta, contenido devuelto y estado de los datos), no solo que la petición no falle.
- Incluyen casos de error, y no únicamente casos satisfactorios. Tras una operación rechazada, se verifica que el sistema no ha quedado en un estado inconsistente.

Opcionalmente, el equipo puede añadir **pruebas de extremo a extremo** que automaticen el flujo principal en un navegador real, o pruebas unitarias para la lógica de negocio más compleja. Estas pruebas se valorarán como un complemento, pero no sustituyen a las pruebas de API o de integración exigidas.

## Calidad y experiencia de uso

- La navegación debe ser coherente y permitir entender dónde se encuentra el usuario y qué acciones puede realizar.
- Las operaciones que puedan tardar deberán ofrecer una indicación de progreso o de espera cuando sea pertinente.
- Las acciones destructivas o difíciles de deshacer deberán tener un comportamiento deliberado y comprensible.

- El código deberá estar organizado y tener nombres comprensibles. Se evitarán duplicaciones innecesarias y secretos incrustados en el repositorio.
- El proyecto incluirá instrucciones para instalar, configurar y ejecutar la aplicación, así como las credenciales de prueba necesarias si procede (nunca contraseñas reales).

# Tecnologías

No se impone un lenguaje de programación ni un framework concreto para el servidor. Se puede elegir cualquier tecnología adecuada para desarrollar la aplicación, siempre que se cumplan estas condiciones:

- Se utiliza JavaScript; no se permite WebAssembly.
- El uso de librerias de JavaScript es libre, pero será consensuada con el equipo y el profesor (React...)
- La comunicación cliente-servidor se realiza mediante HTTP; no se permite WebSockets.
- La aplicación se puede ejecutar y probar de acuerdo con las instrucciones incluidas en el proyecto.
- Existe una forma de automatizar pruebas contra el servidor mediante HTTP con las herramientas elegidas.
- No se pueden usar plataformas No-code/Low-code que oculten la implementación de la aplicación.
- La base de datos tiene que estar contenida en el proyecto y no depender de servicios externos, salvo que se acuerde previamente (Supabase).

Se puede utilizar una arquitectura con renderizado en servidor, una aplicación cliente que consuma una API HTTP, o una combinación de ambas.

# Evaluación

**El sistema de evaluación está pendiente de definir.** Una posibilidad es combinar varias evidencias para valorar tanto el funcionamiento de la aplicación como la comprensión individual del trabajo realizado. Por ejemplo:

1. **Vídeo de presentación.** El equipo grabaría un vídeo breve mostrando el funcionamiento de la aplicación. Éste incluiría al menos un recorrido principal satisfactorio (*happy path*) para cada rol de usuario y el tratamiento de algún error, como un formulario inválido, un permiso insuficiente o una operación que no se puede completar.
2. **Entrega del código fuente.** Se revisaría el repositorio para contrastar la implementación con lo mostrado en el vídeo y con los requisitos de la práctica.
3. **Auditoría.** Cada alumno realizará una auditoría de la aplicación de otro equipo, comprobando que cumple los requisitos, evaluando la calidad de la aplicación y de su seguridad. La auditoría servirá para calificar tanto al proyecto auditado como al alumno que la realiza.
4. **Examen oral individual.** Cada estudiante respondería preguntas sobre el vídeo, las decisiones de diseño, la arquitectura, el modelo de datos, el código y los resultados de la auditoría. El objetivo sería comprobar que comprende y puede justificar el trabajo presentado por el equipo.

Esta propuesta es orientativa y no constituye todavía una rúbrica definitiva. Los criterios, el peso de cada evidencia, los plazos y el formato de entrega se comunicarán cuando se concrete el sistema de evaluación.

# Ejemplos de proyectos

Las siguientes propuestas son ejemplos de distinta índole. No es necesario elegir una de ellas: se pueden adaptar, combinar o utilizar como inspiración para una idea propia.

## Kahoot! o plataforma de concursos

Una herramienta para crear cuestionarios de preguntas y respuestas, organizar partidas y consultar resultados. Un usuario puede crear cuestionarios, editar preguntas y respuestas, compartir una sala y participar en concursos. El sistema registra puntuaciones y muestra clasificaciones. La interacción puede realizarse mediante peticiones HTTP periódicas, sin utilizar WebSockets.

*Entidades posibles:* usuarios, cuestionarios, preguntas, respuestas, partidas, participantes y puntuaciones.

## Gestor de fútbol fantástico

Los usuarios crean equipos virtuales con un presupuesto limitado, seleccionan jugadores, realizan cambios y compiten en ligas privadas. Las puntuaciones pueden introducirse manualmente o calcularse a partir de datos de una fuente externa si el alcance lo permite.

*Entidades posibles:* usuarios, equipos, jugadores, ligas, alineaciones, fichajes y jornadas.

## Juego de navegador por turnos

Una experiencia inspirada en los juegos de navegador clásicos: cada usuario crea un personaje, mejora sus atributos, consigue objetos y desafía a otros jugadores en combates asíncronos. Las acciones se resuelven en el servidor y el resultado queda guardado para futuras visitas.

*Entidades posibles:* usuarios, personajes, atributos, objetos, inventarios, desafíos y resultados.

## Gestor de clubes deportivos

Una plataforma para crear clubes, organizar equipos, gestionar miembros, publicar entrenamientos y coordinar partidos o torneos. Puede incluir inscripciones y permisos diferentes para organizadores y participantes.

*Entidades posibles:* usuarios, clubes, equipos, miembros, eventos, inscripciones y resultados.

## Planificador colaborativo de viajes

Los usuarios crean viajes, invitan a otras personas, proponen destinos, votan actividades y organizan un itinerario compartido. La aplicación mantiene el historial de propuestas y permite modificar el plan de forma colaborativa.

*Entidades posibles:* usuarios, viajes, participantes, destinos, actividades, votos e itinerarios.

## Mercado de segunda mano para estudiantes

Una plataforma para publicar artículos, buscar productos, guardar favoritos y contactar con sus propietarios. Se pueden añadir ofertas, reservas y estados de disponibilidad, sin necesidad de implementar pagos reales.

*Entidades posibles:* usuarios, anuncios, categorías, favoritos, ofertas, conversaciones y reservas.

## Comunidad de libros, películas o series

Los usuarios catalogan obras, crean listas, publican reseñas y valoraciones, siguen a otros usuarios y comparten recomendaciones. La aplicación puede generar listados según las preferencias o la actividad

de la comunidad.

*Entidades posibles:* usuarios, obras, listas, reseñas, valoraciones, etiquetas y relaciones de seguimiento.

## Intercambio de habilidades entre estudiantes

Los usuarios ofrecen habilidades que pueden enseñar y publican aquello que desean aprender. La aplicación facilita encontrar coincidencias, solicitar sesiones, acordar horarios y valorar la experiencia después de la sesión.

*Entidades posibles:* usuarios, habilidades, ofertas, solicitudes, sesiones, disponibilidad y valoraciones.

## Organizador de eventos y comunidades

Los usuarios crean grupos o clubes, publican eventos, gestionan inscripciones y consultan quién asistirá. Los organizadores pueden modificar los eventos y gestionar las plazas disponibles.

*Entidades posibles:* usuarios, grupos, eventos, lugares, inscripciones, listas de espera y comentarios.

## Gestor de colecciones personales

Una aplicación para catalogar colecciones de libros, discos, cómics, videojuegos, camisetas u otros objetos. Permite añadir imágenes y atributos personalizados, organizar elementos en colecciones, mantener listas de deseos y compartir el catálogo.

*Entidades posibles:* usuarios, colecciones, elementos, categorías, imágenes, listas de deseos e intercambios.

## Plataforma de recetas y planificación de comidas

Los usuarios publican recetas, organizan colecciones, preparan menús semanales y generan listas de la compra. También pueden valorar recetas y proponer modificaciones.

*Entidades posibles:* usuarios, recetas, ingredientes, pasos, menús, listas de la compra y valoraciones.

## Gestor de ligas de juegos de mesa

Los usuarios crean grupos, registran partidas, organizan torneos y consultan estadísticas históricas. El sistema calcula clasificaciones según las reglas elegidas por cada grupo.

*Entidades posibles:* usuarios, grupos, juegos, partidas, participantes, torneos y clasificaciones.

## Plataforma de voluntariado

Las organizaciones publican oportunidades de voluntariado y los usuarios se inscriben según sus intereses y disponibilidad. Los responsables pueden gestionar solicitudes, turnos y participación.

*Entidades posibles:* usuarios, organizaciones, oportunidades, solicitudes, turnos, inscripciones y habilidades.

## Adopción de mascotas y voluntariado animal

Las protectoras publican perfiles de animales, gestionan solicitudes de adopción y coordinan voluntarios. Los usuarios pueden guardar animales favoritos y consultar el estado de sus solicitudes.

*Entidades posibles:* usuarios, protectoras, animales, solicitudes, citas, turnos y actividades.

## Gestor de finanzas personales con retos

Los usuarios registran gastos e ingresos, definen presupuestos y establecen objetivos de ahorro. Se pueden añadir retos voluntarios y estadísticas de progreso, evitando publicar información financiera

privada de los usuarios.

*Entidades posibles:* usuarios, cuentas, categorías, movimientos, presupuestos, objetivos y retos.

## Gestor de restaurante o negocio virtual

Cada usuario administra un negocio ficticio: define productos o menús, organiza recursos, mejora instalaciones y observa indicadores de rendimiento. Los resultados se calculan a partir de reglas de simulación implementadas en el servidor.

*Entidades posibles:* usuarios, negocios, productos, menús, recursos, mejoras y resultados.

# Guía para la definición del proyecto

Puntos tratados en clase para ayudar a los equipos a definir su proyecto.

- Descripción de la idea.
- Usuarios y roles.
- Integración con sistemas externos (IA, correo electrónico, API). Producción y/o consumo.
- Definir una planificación (entregas, hitos).
- Definición de pruebas de aceptación.
- Clasificación de requisitos por áreas funcionales.
  - **Gestión de usuarios:**
    - Almacenamiento de credenciales.
    - Contenido del formulario de alta de usuarios.
    - Cómo se evita el abuso del formulario de alta.
    - Definición de quién asigna roles.
  - **Modelo de datos:**
    - Distinguir entre datos (base de datos) o documentos (sistema de ficheros).
    - Vistas y consultas sobre los datos. Auditabilidad.
    - Evolución temporal de los datos. Ciclo de vida.
  - **Interfaz de usuario:**
    - *Responsive*: ¿*framework* CSS?
    - Aspecto (*look*):
      - Tema claro/oscuro.
      - Ubicación de elementos comunes (consistencia).
      - Guía de estilo.
  - **Requisitos tecnológicos:**
    - Modularidad de la aplicación (servicios).
    - Test de aceptación de usuario (UAT).
    - Pruebas de integración / unitarias / *end-to-end*.